Module 1 : Modélisation Objets Avancée

TP HEP — Gestion d'Étudiants (TDD, Python & Java)

Créer un TP structuré sur l'héritage en programmation orientée objet, avec une démarche **TDD (Test-Driven Development**) rigoureuse (**Red** → **Green** → **Refactor**), en Python puis Java.

Objectifs pédagogiques

1. Héritage : comprendre la relation « est-un » (Etudiant est une Personne)
2. Encapsulation : attributs privés, getters/setters
3. Composition : un Etudiant possède une liste de Cours
4. TDD : écrire les tests *avant* le code, cycle Red → Green → Refactor
5. Contrats : validation des entrées (nom non vide, âge > 0, moyenne ∈ [0, 20])

TP 1 : Héritage & TDD – Application de Gestion Étudiante

Ecrire une application de gestion d'étudiants avec les classes suivantes:

1. **La classe** "Personne" avec les attributs "nom" et "âge",

Elle représente les informations de base pour chaque individu

2. **La classe** "Étudiant" (classe dérivée) hérite de la classe "Personne" et ajoute des attributs spécifiques aux étudiants, tels que le numéro d'étudiant, la moyenne et une liste de cours.

3. **La classe** "Cours" représente les cours proposés. Elle contient des attributs tels que le nom du cours et le professeur responsable.

4. **un programme main** qui sert à créer des instances d'étudiants, ajouter des cours pour chaque étudiant, et affichons leurs détails ainsi que les détails des cours.

TP 2 : Encapsulation – Sécurisation des Données Académiques

Objectif : Maîtriser l'usage des modificateurs d'accès, des getters/setters et la validation des données pour protéger l'intégrité des objets.

Énoncé

Dans votre application de gestion, nous voulons nous assurer que les données sensibles (comme la moyenne) ne soient pas modifiées de manière incohérente.

1. **Modification de la classe Etudiant :** Rendez les attributs nom, age et moyenne privés.
2. **Règles de gestion :** * L'âge ne peut pas être inférieur à 0 ou supérieur à 100.
    - La moyenne doit obligatoirement être comprise entre 0 et 20.
3. **Programme Main :** Tentez de modifier une moyenne avec une valeur invalide (ex: 25) et vérifiez que l'objet refuse la modification ou lève une exception.

TP 3 : Polymorphisme – Extension du Système de Notation

Objectif : Utiliser le polymorphisme pour traiter différents types de membres de l'école de manière uniforme tout en conservant des comportements spécifiques.

Énoncé

L'école souhaite maintenant gérer non seulement les étudiants, mais aussi les Enseignants. Tous deux sont des Personne, mais leur méthode d'affichage est différente.

1. Création de la classe Enseignant : Hérite de Personne et ajoute un attribut salaire.
2. Méthode afficher_details() : Définissez cette méthode dans la classe mère Personne. Redéfinissez-la (Override) dans Etudiant pour afficher sa moyenne, et dans Enseignant pour afficher son salaire.
3. Programme Main : Créez une liste (ou un tableau) de type Personne contenant à la fois des étudiants et des enseignants. Parcourez la liste et appelez afficher_details() pour chaque élément.

« C’est l’effort qui fait les forts »